

# **Project 6:**

## **Image Classification of Dogs & Cats using a Keras Deep Learning Algorithm**

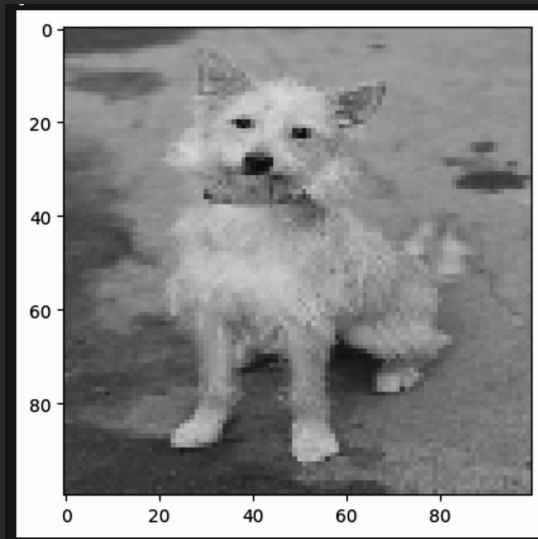
**Presented by:** Raven Otero-Symphony

# Agenda

---

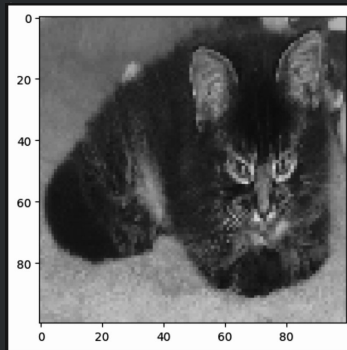
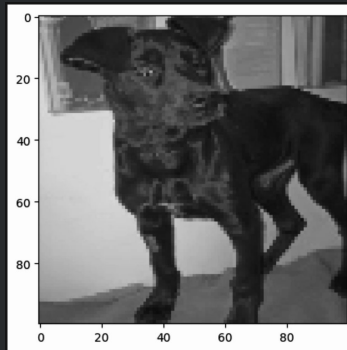
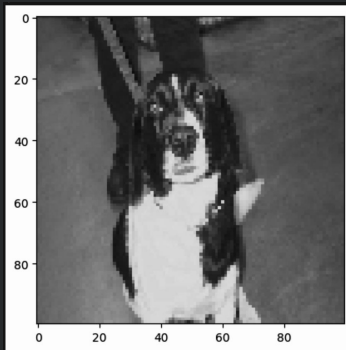
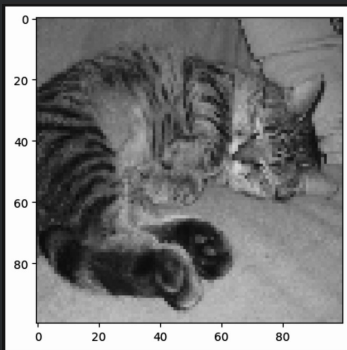
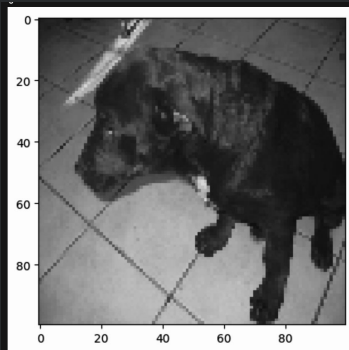
- Project Goals
- Data Processing
- Results
- Real World Test
- Questions

```
Epoch 1/8  
624/624 ————— 19s 24ms/step - accuracy: 0.6584  
Epoch 2/8  
624/624 ————— 10s 16ms/step - accuracy: 0.7504  
Epoch 3/8  
624/624 ————— 10s 16ms/step - accuracy: 0.7938  
Epoch 4/8  
624/624 ————— 10s 16ms/step - accuracy: 0.8223  
Epoch 5/8  
624/624 ————— 10s 16ms/step - accuracy: 0.8449  
Epoch 6/8  
624/624 ————— 10s 16ms/step - accuracy: 0.8700  
Epoch 7/8  
624/624 ————— 10s 16ms/step - accuracy: 0.8912  
Epoch 8/8  
624/624 ————— 10s 16ms/step - accuracy: 0.9127  
<keras.src.callbacks.history.History at 0x7dfde0f8ee40>
```



# Project Goals

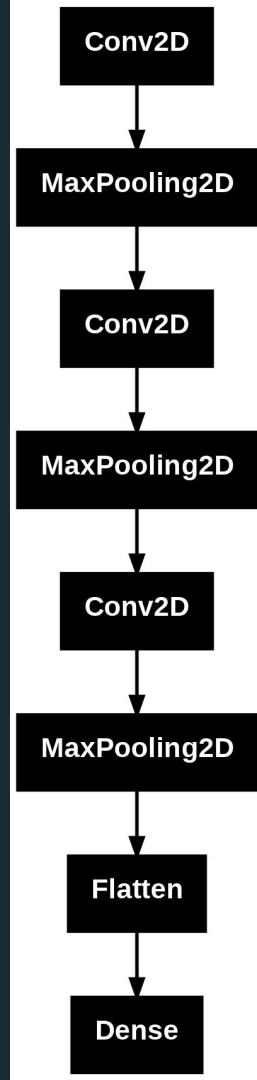
- “Deep Learning” is a combination of data science, machine learning, and artificial intelligence.
- This project builds a Convolutional Neural Network (CNN) to predict whether a grayscale image is a cat or dog.
- Several conditions were required, including: convolutional layers, activation functions, and more.



# Data Processing

- **Model:** Sequential
- **Total Hidden Layers:** 3
  - ◆ **Filters:** 64
  - ◆ **Kernel Size:** (3, 3)
  - ◆ **Activation:** Relu
  - ◆ **Padding:** Same
- **Output Layer Activation:** Sigmoid
- **Model Compiler:** Adam
- **Model Epochs:** 8

```
1 # model selection
2 model = Sequential()
3
4 # Define input layer
5 model.add(
6     Input(
7         shape=X_train.shape[1:],
8         name="model_input",
9     ),
10 )
11
12 # Define first hidden layer
13 model.add(
14     Conv2D(
15         name = "Conv1",
16         filters = 64,
17         kernel_size = (3,3),
18         activation = 'relu',
19         padding='same'
20     )
21 )
22 model.add(MaxPooling2D(pool_size=(2,2)))
23
24 # Define second hidden layer
25 model.add(
26     Conv2D(
27         name = "Conv2",
28         filters = 64,
29         kernel_size = (3,3),
30         activation = 'relu',
31         padding='same'
32     )
33 )
34 model.add(MaxPooling2D(pool_size=(2,2)))
35
36 # Define third hidden layer
37 model.add(
38     Conv2D(
39         name = "Conv3",
40         filters = 64,
41         kernel_size = (3,3),
42         activation = 'relu',
43         padding='same'
44     )
45 )
46 model.add(MaxPooling2D(pool_size=(2,2)))
```



# Results

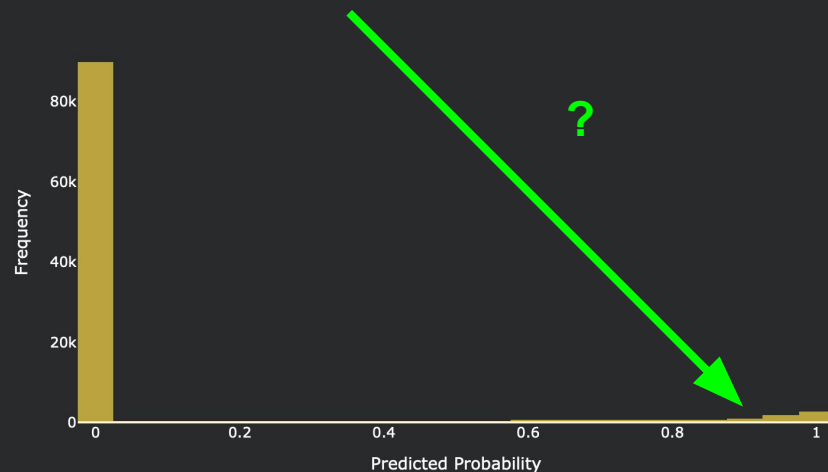
- **Model Accuracy:** 85%
- Unbalanced prediction distribution (0 = dog, 1 = cat)
- Non-zero or one predictions?

Model: "sequential"

| Layer (type)                   | Output Shape         | Param # |
|--------------------------------|----------------------|---------|
| Conv1 (Conv2D)                 | (None, 100, 100, 64) | 640     |
| max_pooling2d (MaxPooling2D)   | (None, 50, 50, 64)   | 0       |
| Conv2 (Conv2D)                 | (None, 50, 50, 64)   | 36,928  |
| max_pooling2d_1 (MaxPooling2D) | (None, 25, 25, 64)   | 0       |
| Conv3 (Conv2D)                 | (None, 25, 25, 64)   | 36,928  |
| max_pooling2d_2 (MaxPooling2D) | (None, 12, 12, 64)   | 0       |
| flatten (Flatten)              | (None, 9216)         | 0       |
| output (Dense)                 | (None, 20)           | 184,340 |

Total params: 776,510 (2.96 MB)  
Trainable params: 258,836 (1011.08 KB)  
Non-trainable params: 0 (0.00 B)  
Optimizer params: 517,674 (1.97 MB)

Aggregate Distribution of Predicted Probabilities

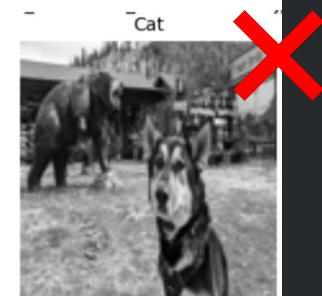
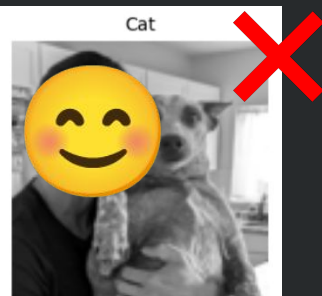
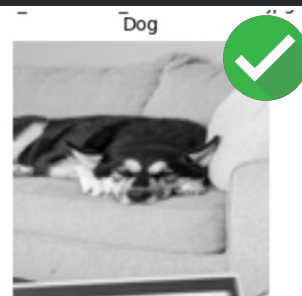
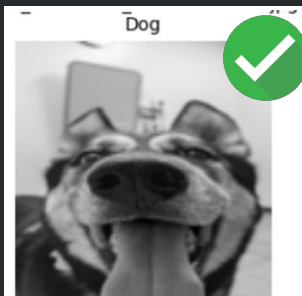
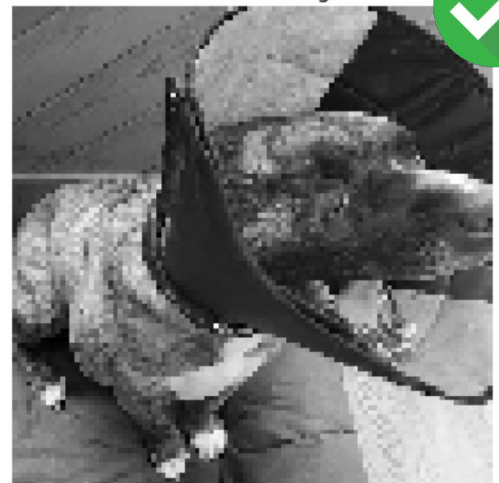


# Real World Test

- Correctly predicted test file as “dog.”
- Incorrectly predicted 4 out of 6 real world images of my dogs.
- **Model Improvement Hypothesis:** Data cleaning, model adjustments, greater image sample

1/1 0s 44ms/step

Prediction: Dog!





# Thank you!

## Questions?



[\[View Github\]](#)